

AtCoder Educational DP Contest

26-Problem ICPC Trick Sheet

States, transitions, and recognition cues for fast contest recall

A — Frog 1

State: $dp[i]$ is the minimum cost to reach stone i . Since jumps are only length 1 or 2, transition from $i - 1$ and $i - 2$ with the corresponding height difference. **Trick:** Basic minimum-cost-on-a-line DP.

B — Frog 2

State: $dp[i]$ is the minimum cost to reach stone i . Use

$$dp[i] = \min_{1 \leq j \leq K} (dp[i - j] + |h_i - h_{i-j}|).$$

Trick: If each position depends on the previous K states, plain $O(NK)$ DP is often enough.

C — Vacation

State: $dp[i][a]$ is the maximum happiness through day i if activity a is chosen today. Transition only from the other two activities. **Trick:** Choose one option per day with forbidden consecutive repeats.

D — Knapsack 1

State: $dp[w]$ is the maximum value achievable with total weight at most w . For each item, iterate weights **backwards** so that the same item is not reused. **Trick:** Canonical 0/1 knapsack when W is small.

E — Knapsack 2

Weight is too large, but total value is small. **State:** $dp[v]$ is the minimum weight needed to achieve total value v . **Trick:** Invert the knapsack dimension; answer the largest v with $dp[v] \leq W$.

F — LCS

State: $dp[i][j]$ is the LCS length of prefixes $s[0..i)$ and $t[0..j)$. If characters match, go diagonally +1; otherwise take the best of dropping one character. **Trick:** Reconstruct by walking backward through the table.

G — Longest Path

Because the graph is a DAG, process vertices in topological order. **State:** $dp[v]$ is the longest path ending at v . Relax every edge with $dp[u] = \max(dp[u], dp[v] + 1)$. **Trick:** DAG longest path becomes simple DP once cycles are gone.

H — Grid 1

State: $dp[i][j]$ counts paths to cell (i, j) . For an open cell, add from top and left; for a wall, set the state to zero. **Trick:** Only the previous row/current left value matters, so compress to $O(W)$ memory.

I — Coins

State: $dp[j]$ is the probability of exactly j heads after the processed coins. Each coin splits every state into a heads and tails transition. **Trick:** At the end sum states with $j > n - j$.

J — Sushi

The exact plate arrangement is irrelevant; only counts (a, b, c) of plates with 1, 2, and 3 sushi matter. Empty plates create a self-loop, giving

$$E = \frac{N + aE_1 + bE_2 + cE_3}{a + b + c}.$$

Trick: Compress symmetric states, then solve the expectation self-loop algebraically.

K — Stones

State: $win[x]$ says whether the current player can force a win

with x stones. A state is winning iff **some** legal move reaches a losing state. **Trick:** Standard impartial-game win/lose DP.

L — Deque

Store the optimal **score difference** instead of both players' totals. For interval $[l, r]$,

$$dp[l][r] = \max(a_l - dp[l + 1][r], a_r - dp[l][r - 1]).$$

Trick: Score-difference DP often removes the need to model both players separately.

M — Candies

State: $dp[j]$ is the number of ways to distribute exactly j candies so far. Each transition sums a contiguous range of previous states. **Trick:** Prefix sums make each range sum $O(1)$, reducing $O(NK^2)$ to $O(NK)$.

N — Slimes

State: $dp[l][r]$ is the minimum cost to merge all slimes in $[l, r]$ into one. Split at every m and use

$$dp[l][r] = \min_m (dp[l][m] + dp[m][r] + \text{sum}(l, r)).$$

Trick: Classic interval DP; prefix sums make merge cost $O(1)$.

O — Matching

Use a bitmask of already-used women; the next man is automatically popcount($mask$). Try every compatible unused woman. **Trick:** Standard assignment/matching DP for $N \lesssim 20$.

P — Independent Set

Tree DP with two states per vertex: white or black. A black vertex forces every child white; a white vertex allows either color. **Trick:** Independent child contributions multiply.

Q — Flowers

This is a weighted LIS. For height h_i , query the best beauty among subsequences ending at a **smaller** height, then add a_i . **Trick:** Use a Fenwick/segment tree storing prefix maximums instead of LIS lengths.

R — Walk

For adjacency matrix A , the entry $(A^K)_{ij}$ counts walks of exactly K edges from i to j . **Trick:** "Count walks of huge exact length" $\Rightarrow$ binary matrix exponentiation; sum all entries of A^K .

S — Digit Sum

Digit DP state is (position, sum mod D , tight). Leading zeros automatically represent shorter numbers. **Trick:** Subtract one at the end to exclude zero.

T — Permutation

Build the relative order one element at a time and let $dp[j]$ track the rank of the newest element among the current prefix. $A <$ constraint takes a prefix sum of ranks and $>$ takes a suffix sum. **Trick:** Cumulative sums reduce the naive $O(N^3)$ transition to $O(N^2)$.

U — Grouping

Precompute $score[mask]$, the total pair contribution if that subset forms one group. Then

$$dp[mask] = \max_{sub \subseteq mask} (score[sub] + dp[mask \setminus sub]).$$

Trick: Force sub to contain one fixed bit of $mask$ to avoid equivalent partitions.

V — Subtree

Compute the contribution from children, then reroot so each node also receives the contribution from its parent side. To send from v to child u , multiply all neighbor contributions except u . **Trick:** Prefix/suffix products avoid modular division, important because the modulus is arbitrary.

W — Intervals

Process positions left to right; each DP state represents the position of the most recent selected 1. An interval $[l, r]$ ending at r contributes exactly to a contiguous range of states. **Trick:** This becomes range-add plus global-max, so use a lazy segment tree.

X — Tower

Sort blocks by $w + s$. Let $dp[x]$ be the best value when total weight x is already above the current block; the block may be placed underneath iff $x \leq s$. **Trick:** After the ordering

argument, the problem becomes 0/1 knapsack.

Y — Grid 2

Without obstacles, monotone path counts are binomial coefficients. Sort blocked cells plus the destination, let $dp[i]$ count valid paths to point i , then subtract paths that arrive through each earlier blocked point. **Trick:** Combinatorics + DP over only the special points.

Z — Frog 3

Expand

$$dp[i] = \min_j (dp[j] + (h_i - h_j)^2 + C)$$

into

$$h_i^2 + C + \min_j ((-2h_j)h_i + dp[j] + h_j^2).$$

Each previous j becomes a line queried at $x = h_i$. **Trick:** Since heights increase, slopes and queries are monotone, so a deque Convex Hull Trick gives $O(N)$.